Luca Danisi **20-4-2026**

Buffer overflow

1 - Objective :

The goal of this lab is to understand what a **Buffer Overflow (BOF)** is and to observe it in action on a small program written in C.

In this lab I will:

1. Write a deliberately vulnerable C program that declares a buffer of 10 characters and reads user input without any length check.
2. Compile and run it normally to verify it works correctly with short inputs.
3. Trigger the Buffer Overflow by providing an input longer than the buffer, causing a segmentation fault.
4. Modify the code by increasing the buffer size to 30 characters, to answer the question: *"Is increasing the buffer size enough to fix the vulnerability?"*

The final goal is not only to reproduce the error, but to understand the **real cause** of the BOF and why increasing the buffer size alone is not a security solution.

2 - 🧪 Our lab enviroment

🐉 **Kali Linux**: Attacker machine (192.168.50.100)

VirtualBox Internal Network

3 – Tool Used :

Tool	Purpose
Terminal (Bash/Zsh)	Entire lab performed from the command line
GCC (GNU Compiler Collection)	Standard C compiler on Linux — translates .c source code into an executable file
Nano	Text editor used to write and modify the C source code

Tool	Purpose
C Language	Programming language used for the deliberately vulnerable program. C is ideal for studying Buffer Overflows because, unlike modern languages, it does not perform automatic bounds checking on buffers — memory management is left entirely to the programmer

4 – Results

✅ Test 1 — Buffer of 10 chars, short input

Wrote the program BOF.c declaring a buffer of 10 characters and compiled it with gcc -g BOF.c -o BOF. On the first run I entered the string esercizio (9 characters).

Result: the program printed Nome utente inserito: esercizio and closed without errors. Everything as expected — 9 characters fit comfortably in a 10-character buffer.

❌ Test 2 — Buffer of 10 chars, long input

I re-ran the same program, this time entering a much longer string: eserciziomoooltopiùlungo (24 characters).

Result: the program still printed Nome utente inserito: eserciziomoooltopiùlungo, but immediately after the error appeared:

zsh: segmentation fault ./BOF

This is the classic sign of a Buffer Overflow: the extra characters overwrote unauthorized memory areas and the operating system terminated the process to protect itself.

✅ Test 3 — Buffer of 30 chars, short input

I modified the source code changing char buffer [10]; to char buffer [30];, recompiled with gcc -g BOF.c -o BOF and re-ran the program with the same esercizio string.

Result: everything worked fine, the program responded correctly without errors.

❌ Test 4 — Buffer of 30 chars, long input

With the 30-char buffer, I entered an even longer string: eserciziomooooltopiulungodiquellodiprima (40 characters).

Result:

5 - Issues Encountered During the Test

No major technical issues were encountered during the lab.

The main point that required attention was choosing input strings long enough to reliably trigger the segmentation fault — especially after increasing the buffer size to 30 characters, where shorter overflow attempts did not always crash the program.

6 - Conclusion

Increasing the buffer size does not fix the vulnerability — it only moves it. It raises the threshold beyond which the problem appears, but the root cause remains exactly the same.

The real cause is not the size of the buffer, but the fact that the program reads user input without any length check. As long as the program accepts any amount of characters without verifying if there is enough space, an attacker can always provide an input longer than the buffer and trigger an overflow.

To truly eliminate the vulnerability, the input must be validated before being written into the buffer, by defining the maximum number of characters accepted. Safer alternatives to scanf("%s", buffer) include fgets() with a defined size limit, or scanf("%9s", buffer) to enforce a hard limit at read time.

Security Takeaways

- Never trust user input — always validate length and content before processing
- Bounds checking is the programmer's job in C — the language won't do it for you
- Buffer overflows are the foundation of many critical CVEs (EternalBlue, Heartbleed, BlueKeep) — understanding them at this level is key to recognizing them in real-world vulnerabilities.

Technical walkthrough

Step 1 — Open terminal and move to Desktop

Opened the terminal and moved to the Desktop:

cd /home/kali/Desktop

Step 2 — Create the source file BOF.c

Created the file using the nano text editor:

```
nano BOF.c
```

Wrote the following code:

```
#include <stdio.h>
int main () {
    char buffer [10];
    printf ("Si prega di inserire il nome utente:");
    scanf ("%s", buffer);
    printf ("Nome utente inserito: %s\n", buffer);
    return 0;
}
```

The program declares a 10-character buffer, asks the user for a username through scanf("%s", buffer) — a function that does **not** check input length — and prints it to screen.

Step 3 — Compile the program

Compiled the source code:

```
gcc -g BOF.c -o BOF
```

GCC translates the BOF.c source code into machine code and saves the result as the executable file BOF. The -g flag adds debugging information to the binary.

Step 4 — First test (buffer 10, short input)

Ran the program:

```
./BOF
```

Entered esercizio (9 characters). The program printed the username correctly and exited without errors.

Step 5 — Second test (buffer 10, long input) — Buffer Overflow!

Re-ran `./BOF` and entered a much longer string: eserciziomoooltopiùlungo (24 characters).

Result:

```
zsh: segmentation fault ./BOF
```

The program crashed with a segmentation fault: the extra characters overwrote unauthorized memory areas and the operating system terminated the process.

Step 6 — Modify the code (buffer 30)

Modified the source changing the buffer declaration:

- From: `char buffer [10];`
- To: `char buffer [30];`

New code:

```
#include <stdio.h>
int main () {
    char buffer [30];
    printf ("Si prega di inserire il nome utente:");
    scanf ("%s", buffer);
    printf ("Nome utente inserito: %s\n", buffer);
    return 0;
}
```

Step 7 — Recompile

Since the code was modified, the program was recompiled:

```
gcc -g BOF.c -o BOF
```

Step 8 — Third test (buffer 30, short input)

Ran ***./BOF*** and entered esercizio again. The program worked without errors.

Step 9 — Fourth test (buffer 30, long input) — Buffer Overflow again!

Ran ***./BOF*** and entered an even longer string: eserciziomooooltopiulungodiquellodiprima (~40 characters).

Screenshot

Figure 1 – Vulnerable C program in nano

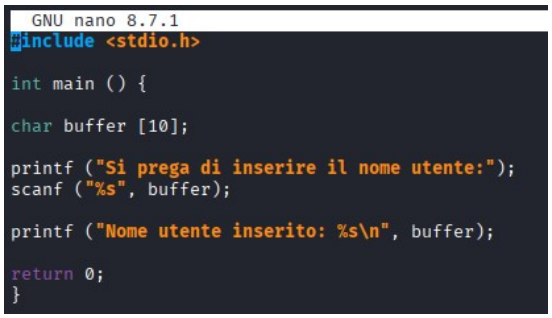

Figure 2 – Buffer overflow trigger